

Revisiting One of Cybersecurity's Most Effective Yet Overlooked Practices

Threat Modelling for the Identification, Prioritisation, and Resolution of Threats, When and Where It Matters Most

By Godfrey Kutumela, CEO and Lead Cybersecurity Practitioner, Nucleus Systems

Cybersecurity has spent decades chasing speed, scale, automation, and intelligence. We have built better scanners, larger vulnerability databases, more advanced detection platforms, richer telemetry pipelines, and increasingly capable artificial intelligence. Yet many organisations still struggle with a very old and very expensive problem: they discover their most serious security weaknesses too late, too late in design, too late in development, too late in deployment, and too often only after an attacker has already found the path of least resistance.

This is why threat modelling deserves a serious reintroduction, not as a compliance checkbox, not as a workshop that produces a diagram no one updates, and not as an abstract exercise owned only by specialists, but as one of the most practical and powerful techniques the industry has for identifying, prioritising, and resolving threats at the precise moment when decisions still matter most.

I. The Problem: Security Arrives After the Critical Decisions Are Made

Many security programmes are still built around late discovery. Code is scanned after it is written. Infrastructure is assessed after deployment. Applications are penetration tested shortly before release. Risk is documented after the architectural decisions are already locked in. The pattern is familiar: security teams identify valid issues, engineering teams struggle to remediate them under deadline pressure, product teams worry about delays, and executives are left managing trade-offs that earlier work could have avoided entirely.

The result is predictable. Some issues are fixed. Some are deferred. Some are accepted without enough context. And some remain invisible until an incident makes them visible to everyone. The deeper truth is structural: by the time most security activity begins, the architecture has been frozen, the access model has been provisioned, the integrations have been built, and the runbooks have already been written. From that point forward, security is largely a remediation exercise, conducted at a far higher cost than design would have demanded, against a target that is now both more exposed and harder to change.

Threat modelling changes this pattern by moving security thinking upstream. Instead of waiting to find vulnerabilities in finished systems, it helps teams reason about risk while the system is still being shaped, connecting security to architecture, data flows, identities, trust boundaries, business impact, and operational reality. That is why threat modelling is not, ultimately, a security activity at all.

Threat modelling is not a security task bolted onto delivery. It is a design discipline that decides what gets built.

II. What Threat Modelling Really Is

Threat modelling is a structured approach to understanding how a system can be attacked, misused, disrupted, or exposed, and deciding how to address those risks before they become exploitable weaknesses in production. The modern framing, popularised by the 2020 Threat Modeling Manifesto, distils the entire discipline into four deceptively simple questions:

1. **What are we working on?**
2. **What can go wrong?**
3. **What are we going to do about it?**
4. **Did we do a good enough job?**

Everything else, the frameworks, the diagrams, the acronyms, is structure layered on top of those four questions. A useful model does not simply list theoretical risks. It helps a team understand which assets and data matter most, where trust boundaries lie, who or what can interact with the system, how data moves through the architecture, which attacker goals are realistic, which threats are most important, what mitigations are required, who owns the response, and what residual risk remains.

In short, threat modelling turns uncertainty into structured security decisions and gives architects, developers, security teams, operations, privacy specialists, risk owners, and business stakeholders a shared language for systems that are far too complex to be secured by a single team at the end of the process.

III. A Short History: From Security Principles to Practical Engineering

Threat modelling did not appear fully formed. It grew out of secure system design, risk analysis, software-architecture review, and attacker-focused reasoning. Its intellectual foundations reach back to the 1970s, to work such as Saltzer and Schroeder's principles for protecting information in computer systems, least privilege, complete mediation, fail-safe defaults, and economy of mechanism, which shaped how secure systems should be designed but left organisations needing practical ways to apply them.

In the 1990s, attack trees gave defenders a structured way to reason about attacker goals rather than merely cataloguing vulnerabilities; Bruce Schneier popularised them in 1999, building on earlier work, as a reusable method for analysing how an adversary could achieve an objective. That same year, Loren Kohnfelder and Praerit Garg at Microsoft created STRIDE: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege, giving software teams a memorable, repeatable taxonomy they could apply during design.

Through the 2000s, Microsoft institutionalised threat modelling inside its Security Development Lifecycle, turning it from a specialist technique into a mainstream engineering practice. The discipline then broadened: LINDDUN emerged from KU Leuven in 2011 to bring privacy threats into focus; PASTA formalised a seven-stage, risk-centric approach that connected technical analysis to business impact; NIST's data-centric work (SP 800-154) moved threat modelling into formal risk-management language; and MITRE ATT&CK and CAPEC gave teams knowledge bases grounded in real-world adversary behaviour and known attack patterns. The 2020 Manifesto then reframed the whole practice around

its four questions and, crucially, shifted ownership from a security priesthood to the engineering teams themselves.

STRIDE made it accessible. Attack trees made attacker reasoning structured. The SDL made it operational. LINDDUN extended it into privacy, PASTA connected it to business risk, and ATT&CK grounded it in real adversary behaviour.

IV. The Proof: How Microsoft Bet Its Reputation on Threat Modelling

No case study illustrates the value of threat modelling more clearly than the company that helped invent the modern discipline. In the early 2000s, Microsoft's products were a byword for security vulnerabilities. The response was the Trustworthy Computing initiative and, with it, the Security Development Lifecycle — a mandatory, company-wide engineering process in which threat modelling was, and remains, a core element. Microsoft today describes threat modelling as an engineering technique that lets architects identify and mitigate potential security issues early, when they are relatively easy and cost-effective to resolve, and positions it as a routine part of the development lifecycle rather than a one-off review.

The conviction inside Microsoft ran deep. Michael Howard, one of the principal architects of the SDL and co-author of *Writing Secure Code*, captured it in a line that has been quoted ever since: if the company could improve software security in only one way, he said, “we would do threat modeling every day of the week.” Coming from an organisation that had absorbed the full reputational cost of insecure-by-default software, that was not rhetoric. It was a statement of where the leverage actually sat.

Asked what single practice he would keep if he could keep only one, a leader of Microsoft's Security Development Lifecycle answered, in effect, that he would threat model every day of the week.

The results compounded over the following decade and a half. Where a default installation of an early-2000s Windows server shipped with a wide attack surface, numerous services enabled, listening, and exploitable out of the box, the post-SDL philosophy of “secure by default” steadily closed that surface.

By one figure cited in retrospectives of the programme, the count of default, out-of-the-box vulnerabilities fell from on the order of several thousand in the Windows 2000 era to a mere handful by the time of Windows 10 roughly a decade into the SDL, and even several of those residual findings were arguably artefacts rather than true defects, such as SMB signing being configured as self-signed rather than enforced on a default installation, a hardening recommendation that vulnerability scanners routinely flag as a medium-severity issue.

The precise numbers matter less than the trajectory: deliberate, design-time reasoning about how the system could be attacked turned the most-targeted software on earth into a dramatically harder target. The mechanism behind that change is worth stating plainly. Threat modelling did not find additional bugs in the finished code; it prevented entire classes of weaknesses from being built in. Services that did not need to run by default were turned off. Trust boundaries that did not need to exist were removed. Privileges that did not need to be granted were withheld. Each of those decisions was made on a whiteboard, not in an incident bridge, and each was orders of magnitude cheaper.

V. Not Only Microsoft: How the Discipline Went Global

If threat modelling were merely a Microsoft preference, it would be a footnote. It is not. It has become an expectation in the most demanding security environments worldwide.

The United States Department of Defense has embedded threat-led thinking into the way it builds software. Its Enterprise DevSecOps guidance treats security as integral to every stage of development under a secure-by-design principle, and in 2025 the Department launched its Cybersecurity Risk Management Construct, which makes “Threat-Informed Assessments” a core tenet and, in its design phase, treats cybersecurity as a design requirement rather than an afterthought.

The U.S. government’s broader secure-by-design agenda, advanced through CISA, and the European Union’s Cyber Resilience Act push in the same direction: designing security in is no longer advice; it is increasingly an obligation, and threat modelling is the most direct mechanism for meeting it.

The standards bodies and the wider industry have followed suit. OWASP positions threat modelling at the heart of its DevSecOps guidance, describing it as a collaborative, repeatable process that should happen as early as possible because the earlier a weakness is found, the cheaper it is to fix. Carnegie Mellon’s Software Engineering Institute teaches teams to use threat modelling to guide not only design but code review, testing, and incident response.

The major cloud providers publish their own threat-modelling guidance for customers building on their platforms, and regulated industries, banking, payments, healthcare, critical infrastructure, increasingly treat a credible threat model as a precondition for trusting a system with real consequences. Adam Shostack, who led SDL threat modelling at Microsoft and wrote the field’s standard text, helped turn what was once tacit expertise into a teachable, widely practised craft.

From the U.S. Department of Defense to the cloud hyperscalers to the global standards bodies, the same conclusion has been reached independently: reason about how the system will be attacked before you build it.

VI. Why Threat Modelling Matters More Now

Threat modelling is not new, but its relevance has sharpened. Modern systems are more distributed, more interconnected, and more dependent on external services than ever before. A single business process may span cloud infrastructure, APIs, SaaS platforms, identity providers, open-source dependencies, CI/CD pipelines, mobile applications, machine identities, third-party integrations, and sensitive data moving across multiple environments and jurisdictions.

In that reality, security cannot rely only on finding known vulnerabilities. Many of the most serious failures come from design weaknesses, trust assumptions, identity misconfigurations, exposed interfaces, excessive privilege, fragile dependencies, and misunderstood data flows, precisely the issues threat modelling is built to expose.

A scanner may detect a missing patch; a penetration test may find an exploitable endpoint; a monitoring platform may surface suspicious activity. Threat modelling asks the prior question: was the system designed in a way that made those problems likely in the first place? It does not replace other controls. It makes them smarter, earlier, and better targeted.

Nothing has sharpened this point quite like the arrival of autonomous, tool-using AI. Agentic systems behave in ways the classic frameworks were never built to capture, they make unpredictable decisions, interact with other agents, and introduce new failure modes such as data poisoning, model extraction, prompt injection, and goal misalignment.

The response has been a new generation of dedicated frameworks. In early 2025, the Cloud Security Alliance introduced MAESTRO (Multi-Agent Environment, Security, Threat, Risk, and Outcome), a layered threat-modelling framework purpose-built for agentic AI that decomposes a system into seven interacting layers, from the foundation models and data operations through the agent frameworks, infrastructure, observability, and security controls, up to the broader agent ecosystem, and reasons about the threats unique to each. The discipline is not a relic. It is actively evolving to meet the newest classes of risk.

VII. The Strategic Value: Identification, Prioritisation, and Resolution

Three outcomes capture where threat modelling creates the most value, and they are worth treating in turn.

1. Identification — finding what matters before attackers do

Generic checklists are not enough because every system has its own architecture, users, data, dependencies, assumptions, and failure modes. A payments platform, a healthcare application, an identity service, an AI system, and an internal admin portal each carry a different threat profile.

Threat modelling forces teams to look at the system in context: Where does sensitive data enter, and where is it stored? Who can access it? Which identities are trusted? What happens if a single service is compromised? What assumptions are being made about internal networks, users, APIs, or third parties? What would an attacker try first? Answering those questions surfaces threats that generic testing reliably misses.

2. Prioritisation — separating noise from material risk

One of the defining problems in cybersecurity is volume. Teams drown in findings, alerts, vulnerabilities, exceptions, and control gaps, and not everything can be fixed at once. Threat modelling prioritises by connecting threats to business impact and system context.

A weakness in an isolated test system is not the same as a design flaw in a production identity service; a missing control around public marketing content is not the same as a weak authorisation model around customer financial data. By ranking threats on impact, exposure, exploitability, affected data, attacker motivation, control strength, detectability, regulatory consequence, and blast radius, an organisation moves from “everything is important” to “these are the risks that matter most.”

3. Resolution — turning analysis into action

Threat modelling creates value only when it leads to decisions and action. A strong model ends not in a catalogue of fears but in concrete outputs: design changes, security requirements, backlog items, compensating controls, test cases, monitoring requirements, or consciously accepted residual risks. In practice, it leads to stronger authentication and authorisation, reduced privilege, encryption of sensitive data, safer API design, improved logging, rate limiting, stronger key management, segmentation, secure deployment patterns, additional detection rules, changed data-retention

practices, or the removal of unnecessary functionality. This is where many organisations fail: they hold a session, document the risks, and lose momentum. A mature approach treats the output as engineering work, not meeting notes, every important threat gets an owner, a decision, and a path to resolution.

A threat model that does not change the design, backlog, test plan, or monitoring baseline is not a security artefact. It is anxiety with diagrams.

VIII. The Methods, and How to Choose Between Them

There is no single method that fits every situation; the right approach depends on the system, the risk profile, the team's maturity, and the purpose of the exercise. The summary below maps the principal methods to their strengths and what to watch for. Mature organisations rarely use one in isolation; they blend a design-centric method such as STRIDE with attacker-goal analysis, privacy modelling, and operational grounding from ATT&CK.

Method	Where it is strongest	What to watch for
STRIDE	Application, API, and cloud-service design reviews built around data-flow diagrams; the most teachable starting point.	Becomes a mechanical checklist if it is not anchored to the real architecture and to business risk.
Attack trees	High-value assets and adversary-goal analysis: how could someone move money, alter the ledger, or exfiltrate records?	Trees grow large quickly; scope to the goals that genuinely matter.
PASTA	Business-critical and regulated systems where security decisions must trace to business impact and threat intelligence.	Heavier; needs facilitation, preparation, and organisational maturity.
LINDDUN	Privacy engineering: linkability, identifiability, unwanted inference, retention, and transparency of personal data.	Complements rather than replaces security-focused modelling.
NIST data-centric	Sensitive, regulated, or mission-critical data is treated as the crown jewel across the systems that touch it.	Less convenient for fast feature teams unless simplified.
MITRE ATT&CK	Detection engineering, SOC operations, purple teaming, and validating that real adversary techniques are covered.	Operational rather than design-focused; pairs with design methods.
CAPEC	Application security: ensuring known attack-pattern classes are not overlooked.	Must be mapped to the actual system, or it becomes another generic list.
MAESTRO	Agentic and multi-agent AI systems, layer by layer, from foundation models to the agent ecosystem.	Emerging, tooling, and practice are still maturing in 2026.

A simple decision rule serves most teams well: for general application design, start with a data-flow diagram and STRIDE; for privacy-heavy systems, add LINDDUN; for business-critical or regulated systems, use PASTA or a risk-centric hybrid; for sensitive-data protection, use the NIST data-centric

approach; for detection and response, use MITRE ATT&CK; for application attack-pattern coverage, use CAPEC; for specific attacker goals, use attack trees; for agentic AI, reach for MAESTRO; and for agile teams, use lightweight story-level modelling for routine work and deeper reviews for high-risk epics.

IX. Where Threat Modelling Delivers the Most Value

Threat modelling is useful wherever something valuable, exposed, or safety-critical is at stake, but it is most powerful where design decisions have long-term consequences.

- **Application and API security.** Applications and APIs remain among the most common sources of design flaws that become incidents. Modelling exposes weak authentication, broken authorisation, insecure data exposure, dangerous workflows, missing abuse protections, unsafe integrations, and excessive trust in client-side controls, and, for API-heavy architectures, issues with identity propagation, tokens, object-level authorisation, and third-party access.
- **Cloud and platform engineering.** Cloud introduces new questions: who can assume which roles, which service identities can access which resources, where secrets are stored, which data stores are exposed, and what the blast radius of a compromised workload is. Threat modelling helps teams reason about identity, network exposure, managed services, infrastructure as code, and deployment pipelines.
- **DevSecOps and CI/CD.** Modern delivery pipelines are powerful and sensitive; if compromised, they become a path straight to production. Modelling examines source-code access, build systems, artefact repositories, secrets, signing, deployment approvals, and dependency management, the machinery that supply-chain attacks increasingly target.
- **Privacy and data protection.** Privacy-focused modelling forces teams to examine data collection and processing in context: what personal data is collected and why, where it is stored, who can access it, how long it is retained, whether it can be linked or re-identified, and whether the individual is aware. This moves organisations from compliance paperwork to genuine privacy engineering.
- **Detection and response.** A good model does not stop at prevention. For the threats that matter, it asks whether the event would be detected, whether logs would show it, whether alerts would fire, who would respond, and what containment would look like, connecting architecture to operations, which is the essence of cyber resilience.
- **Governance, risk, and compliance.** For boards and risk leaders, threat modelling provides traceability: evidence that risks were considered, decisions were made, controls were selected, and residual risk was understood and accepted by accountable owners, valuable for audits, regulatory engagement, board reporting, and customer trust.

X. Why Threat Modelling Often Fails

Despite its value, threat modelling frequently fails to become a sustainable practice. The reason is rarely the method; it is almost always how the organisation implements it.

- **It is done too late.** Performed shortly before release, it becomes a security review under deadline pressure, when architectural change is expensive and politically difficult. It works best early enough to influence design.
- **It is treated as a document, not a decision process.** A polished diagram, a completed template, or a threat register is not the goal. Better security decisions are. A model that does not influence design, backlog, testing, monitoring, or risk acceptance has limited value.
- **It is owned only by security.** Security can facilitate, but developers understand implementation reality, architects understand structure, product owners understand intent, and operations understand failure modes. The practice works best as a collaboration.
- **It becomes too complex.** When every feature triggers a multi-hour workshop, teams quietly abandon the practice. Modelling should be risk-based: lightweight for low-risk changes, more extensive for high-risk systems.
- **Its outputs are not tracked.** If mitigations are not converted into work items with owners, they disappear. Every significant threat should lead to one of four outcomes: mitigate, transfer, accept, or avoid by changing the design.
- **The models go stale.** Systems change new APIs, shifted data flows, expanded cloud permissions, evolving business logic. A model never updated becomes a historical artefact. It should be refreshed whenever identity, sensitive data, external exposure, integrations, or critical workflows change materially.

XI. A Practical Model for Adoption

Organisations that want to strengthen threat modelling should resist the urge to build a perfect programme on day one. A pragmatic adoption path delivers value faster and survives contact with reality.

5. **Start with high-value systems.** Begin where impact is highest, customer-facing applications, identity platforms, payment systems, sensitive-data platforms, cloud landing zones, CI/CD pipelines, critical APIs, AI and automation systems, and major third-party integrations, to create visible value quickly.
6. **Use lightweight, repeatable methods.** For most teams, start with data-flow diagrams and STRIDE; add attack trees for critical attacker goals, LINDDUN for privacy-heavy systems, and ATT&CK for detection coverage. The goal is useful discovery and decision-making, not methodological purity.
7. **Integrate into existing workflows.** Trigger modelling from real events: new product design, major architecture changes, new sensitive data flows, new external integrations, significant cloud changes, high-risk user stories, regulatory requirements, and lessons from incidents. This makes it part of how work happens rather than an extra ceremony.

8. **Produce actionable outputs.** Every exercise should yield key threats, prioritised risks, required mitigations, backlog items, owners, due dates, validation requirements, and explicit residual-risk decisions.
9. **Build reusable patterns.** Over time, develop reference threat models for common architectures, web applications, APIs, microservices, cloud workloads, data pipelines, mobile apps, SaaS integrations, identity flows, and AI-enabled systems, to reduce friction and scale the practice.
10. **Measure maturity by outcomes.** Track the share of high-risk systems modelled, design changes made before build, time to resolve findings, reduction in repeated design flaws, mitigations validated, and the linkage between models and both test cases and detection logic. The strongest measure is whether modelling prevents expensive late-stage findings and reduces real-world exposure.

XII. Re-Introducing Threat Modelling as a Leadership Practice

The industry does not need threat modelling to be rediscovered as an old methodology. It needs threat modelling to be reintroduced as a modern leadership practice, which requires a different positioning. It should not be sold as another security task; it should be understood as a way to make better business and engineering decisions under uncertainty.

For the CISO, it moves security earlier into the lifecycle. For the architect, it validates design assumptions. For the developer, it clarifies what must be protected and why. For the product leader, it reduces late-stage surprises. For the risk leader, it creates evidence of informed decision-making. For the executive, it connects cyber risk to business impact.

This is precisely the shift Nucleus Systems advocates and operationalises. Across our work, in cybersecurity maturity measurement under NS-CMMF, in AI governance and AI threat modelling under NS-AIGF, and in software and supply-chain trust under our Code Trust Assurance Framework, threat modelling is treated not as a one-time artefact but as a living instrument of digital trust: maintained, owned by the people who build and run systems, fed continuously into design, testing, and monitoring, and extended with newer frameworks such as MAESTRO for the AI era. The aim is to help organisations stop asking only, “are we vulnerable?” and start asking, “are we designing systems that are resilient against the threats that actually matter?”

Threat modelling helps organisations stop asking only, ‘are we vulnerable?’ and start asking, ‘are we designing systems that are resilient against the threats that matter most?’

This is The Right Technique at the Right Time

Threat modelling has been part of cybersecurity for decades, yet it remains underused, misunderstood, or poorly operationalised in many organisations. That is a missed opportunity. Few techniques are as effective at bringing security into the moments where it can still shape outcomes; few are as useful for connecting architecture, engineering, operations, privacy, and risk; and few are as practical for identifying and prioritising threats before they become incidents.

Microsoft bet its reputation on it and turned the most-attacked software on earth into a far harder target. The U.S. Department of Defense built it into its software development process. The standards bodies place it at the centre of secure development. And as autonomous AI reshapes the attack surface again, the discipline is evolving rather than retiring.

The industry does not need to treat threat modelling as a legacy practice. It needs to reintroduce it as a modern discipline for complex digital systems, not because it predicts every attack, and not because it replaces testing, monitoring, or incident response, but because it helps teams make better security decisions when and where those decisions matter most.

APPENDIX

Threat Modelling Defined and Explained — A Reference for Practitioners

A.1 Definition

Threat modelling is a security engineering practice used to identify, reason about, prioritise, and mitigate threats before they become exploitable weaknesses in production. OWASP describes it as a repeatable process for modelling a system from a security perspective, identifying applicable threats, and deciding on responses, ideally early in the software development lifecycle and maintained as the system changes.

NIST frames it as a form of risk assessment that models both attacker and defender perspectives around an entity such as data, an application, a host, a system, or an environment; its SP 800-154 work focuses particularly on data-centric threat modelling, where the protected object is selected data within a system.

The United Kingdom's National Cyber Security Centre describes it as the set of techniques used to model and analyse a technology system in order to understand how it might be attacked or otherwise fail, and the controls needed to manage that risk.

A.2 The Four Questions

Whatever method is used, a sound threat model answers four questions: What are we working on? What can go wrong? What are we going to do about it? And did we do a good enough job? These questions, set out in the 2020 Threat Modeling Manifesto, are the foundation on which every framework and diagram is built.

A.3 A Brief Historical Timeline

- **1970s** — Secure system design principles (Saltzer and Schroeder): least privilege, complete mediation, fail-safe defaults, economy of mechanism.
- **1990s** — Attack trees formalise attacker-goal reasoning; popularised by Bruce Schneier in 1999, building on earlier work.
- **1999** — STRIDE was created by Loren Kohnfelder and Praerit Garg at Microsoft.
- **2000s** — Microsoft institutionalises threat modelling within its Security Development Lifecycle.
- **2011** — LINDDUN privacy threat modelling emerges from KU Leuven.
- **2015 onward** — PASTA formalises a seven-stage, risk-centric approach in Risk Centric Threat Modeling.
- **2016 onward** — NIST SP 800-154 introduces data-centric system threat modelling.
- **2010s–2020s** — MITRE ATT&CK and CAPEC become widely used knowledge bases; the 2020 Threat Modeling Manifesto reframes the discipline.
- **2025** — The Cloud Security Alliance introduces MAESTRO for agentic AI threat modelling.

A.4 What a Good Threat Model Contains

A practical threat model is a decision record, not a diagram. It should include: the system scope and assumptions; an architecture or data-flow diagram; the assets and sensitive data; the trust boundaries; the entry points and external dependencies; the threat list; abuse cases or attacker goals; a risk rating or prioritisation; mitigations with named owners; residual-risk decisions; validation evidence; and a review date with the triggers that will prompt an update. A weak threat model is just a diagram; a strong one changes design, backlog, testing, monitoring, and risk acceptance.

A.5 A Recommended Process

1. **Define scope** — identify the system, feature, data, business process, or integration being modelled, using diagrams, entry points, assets, trust levels, and user or abuse stories.
2. **Model the system** — draw the external entities, processes, data stores, data flows, APIs, identities, trust boundaries, and deployment zones.
3. **Identify threats** — apply STRIDE for design, LINDDUN for privacy, ATT&CK for adversary behaviour, CAPEC for attack patterns, and attack trees for high-value goals.
4. **Prioritise risk** — rank by impact, likelihood, exposure, exploitability, affected data, regulatory consequence, and business criticality.
5. **Select mitigations** — stronger authentication and authorisation, input validation, encryption, segmentation, rate limiting, secure defaults, logging, monitoring, key management, safer retention, or removal of risky functionality.
6. **Validate** — confirm that mitigations are implemented and tested, and that unacceptable risks are addressed before release.
7. **Maintain continuously** — update the model as architecture, dependencies, data flows, identities, deployment patterns, or the threat landscape changes.

A.6 Method Selection at a Glance

For general application design, start with a data-flow diagram and STRIDE. For privacy-heavy systems, add LINDDUN. For business-critical or regulated systems, use PASTA or a risk-centric hybrid. For sensitive data protection, use the NIST data-centric approach. For SOC and detection engineering, use MITRE ATT&CK. For application attack-pattern coverage, use CAPEC. For specific attacker goals, use attack trees. For agentic AI, use MAESTRO. For agile teams, combine lightweight story-level modelling with deeper reviews for high-risk epics.

A.7 Final Assessment

Threat modelling is most valuable when it is early enough to influence design, specific enough to reflect the real system, collaborative enough to include developers, architects, security, operations, product, and risk owners, actionable enough to produce backlog items and control decisions, and continuous enough to evolve with the system. The lesson of its history is that it has moved from expert-driven security analysis to a broad engineering discipline — and, in the AI era, it is broadening again.

Contributing to a world where trust is no longer assumed. It is engineered, measured, and continuously proven.